

4 OPERATORS

In programming, operators are symbols or keywords that represent certain actions to be taken on data. They are used to perform mathematical, logical, and other operations on values and variables in a program. Python, as a versatile programming language, provides a wide range of operators for different types of data and operations. The use of operators is fundamental to programming, and understanding how to use them correctly is essential for developing efficient and effective code. Python operators come in various types, such as arithmetic, comparison, logical, bitwise, and more. Each type of operator has its own set of rules and syntax that govern its usage in a program.

In this chapter, we will explore the different types of operators available in Python and how to use them. We will also delve into the importance of operators in programming and how they improve the efficiency and readability of code. Additionally, we will cover practical examples of using operators in Python and how they can be used in real-world applications.

Python provides a variety of operators, such as arithmetic, comparison, assignment, logical, bitwise, membership, and identity. Table 4.1 lists the importance of each of them, and later in the subsequent sections, we have covered the detailed working process with coding examples.

Table 4.1 Importance and types of Python Operators

Operator Types	Importance	Example
Arithmetic	Perform basic arithmetic operations on numbers.	$a + b, c * d, e / f, a - b$
Comparison	Compare two values and return a Boolean value based on the comparison.	$a > b, a == b, c! = d, e > f, g <= f$
Logical	Combine multiple conditions and return a Boolean value based on the combined conditions.	$a \text{ and } b, c \text{ or } d, \text{not } e$
Bitwise	Used to manipulate binary data and perform bitwise operations	$a \& b, a b, a \gg 2$
Assignment	Used to assign values to variables and modify their values	$a = b, c += d, e -= f, g *= h, o \&= p$
Membership	Used to determine whether a value is present in a sequence or not.	$a \text{ in } b, c \text{ not in } d$
Identity	Used to determine whether two variables refer to the same object type or not	$a \text{ is } b, c \text{ is not } d$

4.1 Arithmetic Operators

In Python, there are nine arithmetic operators broadly categorised into two groups, such as unary and binary depending upon the number of operands required. Unary operators are a special case of arithmetic operators that operate on a single operand. In Python, there are two unary arithmetic operators:

- **Unary plus (+):** This operator simply returns the value of its operand, which can be a positive or negative number. It is rarely used since it does not change the value of its operand.
- **Unary minus (-):** This operator negates the value of its operand, effectively changing the sign of the number. It can be used to represent negative numbers or to perform operations such as subtraction by adding the negated value.

```
1 >>>a = 10
2 >>>b = +a #unary plus
3 >>>print(b)
4 10
5 >>>c = -a #unary minus
6 >>>print(c)
7 -10
```

In the second line, the unary plus (+) operator is used to assign the value of **a** to **b**. Since + is a unary operator in this case, it simply returns the value of **a** as it is, without changing its sign. Therefore, the value of **b** is also 10. In the fifth line, the unary minus (-) operator is used to assign the negation of variable **a** to **c**. This means that the value of **c** is -10, but the value of **a**, remain unchanged.

The use of unary operators is particularly useful when dealing with arithmetic operations on variables. For example, the unary minus operator can be used to invert the sign of a variable and unary '+' operator is less commonly used but can be used to enforce positive values in certain calculations.

In Python, there are seven binary arithmetic operators that can be used to perform basic arithmetic operations on numeric data types such as integers, floating-point numbers, and complex numbers. The details of each one of them is listed in Table 4.2.

Table 4.2 List of binary arithmetic operators

Operator	Purpose	Description
+	Addition	It is used to add two operands. For example, if $a = 20$ and $b = 10$, then $c = a + b$, which results in 30.
-	Subtraction	It is used to subtract the second operand from the first operand. If the first operand is less than the second operand, the result is negative. For example, if $a = 20$ and $b = 10$, then $c = a - b$, which is 10.
/	Division	It returns the quotient after dividing the first operand by the second operand. For example, if $a = 25$ and $b = 10$, then $c = a / b$ results in 2.50 as a floating-point type.
*	Multiplication	It is used to multiply one operand with the other. For example, if $a = 20$ and $b = 10$, then $c = a * b$ would be 200.
%	Modulo	It returns the remainder after dividing the first operand by the second operand. For example, if $a = 21$ and $b = 10$, then $c = a \% b$ will be 1.

**	Exponent	It is an exponent operator represented as it calculates the first operand raised to the power of the second operand. For example, if $a = 2$ and $b = 3$, then $c = a ** b$ results in 8.
//	floor division also called integer division	It gives the floor value of the quotient produced by dividing the two operands. For example, if $a = 25$ and $b = 10$, then $a // b$ returns 2 as an integer.

See the example listed in code Listing 4. 1 and suggesting you write the complete code in Python interpreter to understand the behavior of arithmetic operators.

Listing 4. 1 Sample program of arithmetic operators

```
>>>a = 10
>>>b = 5
>>>print(a+b) #addition
15
>>>print(a-b) #subtraction
5
>>>print(b-a) #subtraction
-5
>>>print(a*b) #multiplication
50
>>>print(a/b) #division
2.0
>>>print(a//b) #floor division
2
>>>print(a**b) #exponent
100000
>>>print(a%b) #remainder
0
```

The result of standard division (a/b) is always a floating-point type, even if the dividend is evenly divisible by the divisor. See the example given below:

```
>>>a = 12
>>>b = 3
>>>print(a/b, type(a/b))
4.0 <class 'float'>
```

Floor division ($//$) returns the largest integer that is less than or equal to the quotient of the division. When the result of floor division is positive, the result is as though the fractional part of the quotient is truncated, leaving only the integer part. When the result is negative, the result is rounded towards negative infinity, meaning it is rounded down to the next largest (more negative) integer. See the example given below:

```
>>>a = -25
>>>b = 10
>>>print(a//b)
-3
```

In the above example, $a//b$ returns -3, as -2.5 rounds down to -3 as more negative. The programming examples 4.1 to 4.3 will add clearer idea to it.

Program 4.1: Write a program to take two integers as input and using arithmetic operator swap their values without using third variable.

```

1 # Take input from the user
2 a = int(input("Enter the first integer: "))
3 b = int(input("Enter the second integer: "))
4
5 # Print the original values
6 print("Original values: a = ", a, " and b = ", b)
7
8 # Swap the values without using a third variable
9 a = a + b
10 b = a - b
11 a = a - b
12
13 # Print the swapped values
14 print("Swapped values: a = ", a, " and b = ", b)

```

In this program, first two integers are taken as input from the user (line 2 and 3) using `input()` function and store them in variables `a` and `b`. Then original values of `a` and `b` are printed using the `print()` function at line 6. Next, their values are swapped without using any third variable and exchanged through subtraction and addition operations (line 9-10).

First, the values of `a` and `b` are added and saved it in `a`. Then, the value of `b` is subtracted from `a`, and store it in `b`. Finally, we subtract `b` from '`a`' again and store the result in `a`. At last, the swapped values of `a` and `b` is printed using the `print()` function.

Program 4.2: A program to calculate the Semester Grade Point Average in a 10 point scale by consider the following steps:

- The number of subjects is 3 with credits 2, 3, and 3.
- Take input of the grade of three subjects that you have scored out of a 10 point scale.
- Multiply the grade of each subject with its credit and add them.
- Add the credit of each subject.
- Finally, divide the total score by the total credits to get the Semester Grade Point Average.

```

1 # Taking input of the grade of three subjects out of 10 point scale
2 subject1_grade = float(input("Enter grade of subject 1 of 10: "))
3 subject2_grade = float(input("Enter grade of subject 2 of 10: "))
4 subject3_grade = float(input("Enter grade of subject 3 of 10: "))
5
6 # Multiplying each subject grade with its credit
7 subject1_score = subject1_grade * 2
8 subject2_score = subject2_grade * 3
9 subject3_score = subject3_grade * 3
10
11 # Calculating total score and total credit
12 total_score = subject1_score + subject2_score + subject3_score
13 total_credit = 2 + 3 + 3
14
15 # Calculating Semester Grade Point Average
16 sgpa = total_score / total_credit
17
18 # Printing the SGPA
19 print("Your SGPA is:", round(sgpa, 2))

```

Program 4.3: Write a program that takes the principal amount, rate of interest, and tenure of the loan in months as input from the user, and calculates the simple interest and total amount to pay.

```

1 # taking input from user
2 principal = float(input("Enter the principal amount: "))
3 rate = float(input("Enter the rate of interest: "))
4 tenure = int(input("Enter the loan tenure in months: "))
5
6 # calculate simple interest
7 interest = (principal * rate * tenure) / 100
8
9 # calculate total amount to pay
10 total_amount = principal + interest
11
12 # display the results
13 print("Simple interest = ", interest)
14 print("Total amount to pay = ", total_amount)

```

First, input is taken from user for the principal amount, rate of interest, and loan tenure in months using the `input()` function (at line 2-4). Then simple interest is calculated (at Line 7) using the formula $(P * R * T) / 100$, where **P** is the principal amount, **R** is the rate of interest, and **T** is the loan tenure in months and save this value in the variable `interest`. Later at Line 10, `total_amount` is calculated by adding principal and interest and displayed the results to the user at Line 13-14.

Program 4.4: Write a program to calculate the compound interest on given principal amount (P), rate of interest (R) and time (T). The compound interest formula, $C.I = P \left(1 + \frac{r}{100}\right)^n - P$, can be used, where **n** represents number of times interest is compounded in a year. Calculate the amount to be repaid on a loan of Rs. 12000 for one and half years at 10% per annum compounded half yearly?

```

p = int(input('Enter Principla Amount:'))
r = float(input('Enter rate of interest:'))
t = int(input('Enter time in months:'))
#duration of compounding interest
com_in_months =int(input('Enter Compounding period:'))
#re-calculating roi based on compounding time
roi_in_com_in_months = com_in_months/12 * r
n = t/com_in_months

total_amount = round(p * ((1+(roi_in_com_in_months/100))**n),2)
print(f'Total Amount to be paid: {total_amount}')
print(f'Interest on Principal: {total_amount-p}')

```

```

Enter Principla Amount:12000
Enter rate of interest:10
Enter time in months:18
Enter Compounding period:6
Total Amount to be paid: 13891.5
Interest on Principal: 1891.5

```

Program 4.5: Write a program to take input an integer n and calculate the expression $n + n^2 + n^3$.

```
1 n = int(input("Enter a number: "))
2 result = n + n**2 + n**3
3 print("Result: ", result)
```

To calculate the result of the expression $n + n^2 + n^3$ the exponent operator ****** is used to raise n to the power of 2 and 3 respectively at line 2, and add the results together. In Python, besides the numeric data types, arithmetic operators can also be used with certain other object types, listed in Table 4.3:

Table 4.3 Use of arithmetic operators with other object types

Object Type	Arithmetic Operator	Purpose
list, tuple, string	+	Can be used to concatenate two objects of same sequence type. For example, <i>"hello" + "all"</i> , results <i>"helloall"</i> .
list, tuple, string	*	Can be used to repeat a sequence a certain number of times. For example, <i>list1 * 2</i> , where list1 is an object of list type.
Set	-	Can be used to find the difference between two sets.
Boolean	+, -, *, %, /, //, **	All the binary arithmetic operators are applicable and resultant outcome will be of int type.

4.2 Comparison Operator

In programming comparison operators allow us to compare values and determine the relationship between them. This helps in decision-making and control flow in programs. For example, we can use comparison operators to check if a value is equal to, greater than, less than, or not equal to another value, and based on the result, we can take appropriate actions.

Comparison operators are used in conditional statements, loops, and functions to make decisions based on the comparison result. For instance, in an if statement, we can use comparison operators to test whether a condition is true or false and execute certain code based on that result. Similarly, in a loop, we can use comparison operators to check if the loop should continue or terminate based on the value being processed (discussed in Chapter 6).

In summary, comparison operators are essential in programming as they provide the means to compare values, make decisions based on those comparisons, and control the flow of the program. The details of each comparison operators are listed in Table 4.4. Use of each of them is listed in code Listing 4.2.

Table 4.4 Details and working principal of comparison operators

Operators	Description
equal to (=)	Returns True if the two values being compared are equal, and False otherwise.
not equal to (!=)	Returns True if the two values being compared are not equal, and False otherwise.
greater than (>)	Returns True if the first value is greater than the second value, and False otherwise.
less than (<)	Returns True if the first value is less than the second value, and False otherwise.
greater than or equal (>=)	Returns True if the first value is greater than or equal to the second value, and False otherwise.

less than or equal (<=)	Returns True if the first value is less than or equal to the second value, and False otherwise.
-----------------------------------	---

Listing 4.2 Illustration of comparison operators

```
>>>a = 10
>>>b = 5
>>>print(a > b) #True relation
>>>print(a < b) #False relation
>>>print(a >= b) #True relation
>>>print(a <= b) #False relation
>>>print(a == b) #False relation
>>>print(a != b) #True relation
```

In this program, two variables **a**, **b** are initialized with the values 10 and 5, respectively. Each of the comparison operators (**>**, **<**, **>=**, **<=**, **==**, **!=**) are used to compare the values of **a** and **b** and print out the result either as **True** or **False**. The output of the program will be as follows:

```
True
False
True
False
False
True
```

These set of operators always returns the relation between two operands either as **True** or **False** of **bool** type. See the next illustration which has verified this:

```
>>>print(type(10>12))
<class 'bool'>
```

Note: When comparing floating-point values in Python for equality, it's important to keep in mind that due to the way computers represent floating-point numbers, small rounding errors can occur. As a result, two floating-point values that should be equal may not be exactly equal when compared using the **==** operator. See the example given below:

```
>>>x = 1.1
>>>y = 2.2
>>>print((x+y)==3.3)
False
```

Yikes! The internal representations of the constant 3.3 may not exactly equal to result of, **1.1 + 2.2**, so you cannot rely on the value of the expression to compare with exactly to 3.3.

The preferred way to determine whether two floating-point values are “equal”, we need to consider this small rounding errors. One common approach is to use a tolerance or “epsilon” value, which is a small value that represents the maximum allowable difference between the two values for them to still be considered equal. Consider the example in Listing 4.3, demonstrates how to compare two floating-point values for equality in Python, using an epsilon value:

Listing 4.3 Comparing floating-point values

```

1 a = 0.1 + 0.2
2 b = 0.3
3 epsilon = 1e-10 # set a small epsilon value
4
5 # compare a and b for equality using epsilon
6 if abs(a - b) < epsilon:
7     print("a and b are equal")
8 else:
9     print("a and b are not equal")

```

To compare *a* and *b* for equality, we have subtracted *b* from *a* and take the absolute value of the difference using the `abs()` function, and compare it to the `epsilon` value using the `<` operator. If the absolute difference between *a* and *b* is less than the `epsilon` value, we consider them as equal.

Note: The choice of `epsilon` value depends on the specific application and the precision required. A smaller `epsilon` value provides greater precision but may also lead to false negatives, while a larger `epsilon` value provides less precision but may also reduce the likelihood of false negatives.

In Python, comparison operators can also be used with non-numeric object types such as strings, lists, tuples, sets, and dictionaries. The comparison is done based on the lexicographical ordering of the elements in the objects. For example, when comparing strings, Python compares the ASCII values of each character in the strings. When comparing lists, Python compares the elements in the lists from left to right, one by one. Similarly, for tuples, sets, and dictionaries, the comparison is done based on the elements contained in them.

4.3 Logical Operator

The importance of logical operators lies in their ability to simplify complex decision-making processes in programs. By combining multiple conditions, programmers can write more concise and readable code that is easier to understand and maintain. Logical operators also play a crucial role in control flow statements, such as loops and conditional statements (discussed in Chapter 6), by allowing programs to perform different actions based on specific conditions.

Like other programming languages Python also offers three logical operators, such as, **and**, **or** and **not** operators. Out of these logical **and**, **or** operators takes two operands and logical not takes one operand. A simple logical and, or expression can be formed by including any object type, as follows:

operand1 and operand2

operand1 or operand2

In general, expression with comparison operators is considered as operands for logical operators. Like comparison operator's logical operators also evaluate every expression and return the outcome as Boolean True or False. We have summarized the outcome of logical and, or operators in Table 4.5.

Table 4.5 Working of logical and, or operators

Operand1	Operand2	And	or
False	False	False	False
True	False	False	True

False	True	False	True
True	True	True	True

The True and False values of Table 4.5 are the outcomes of any comparison expression or values of any Python object type that can be treated as True and False values. The complete working process of logical and, or operators are explained next with suitable coding examples.

```
1 a, b = 10, 20
2 operand1 = a == b
3 operand2 = a > b
4 print(f'{operand1} and {operand2} => {operand1 and operand2}')
5 print(f'{operand1} or {operand2} => {operand1 or operand2}')
```

False and False => False
False or False => False

In the above code, both the comparison expression at line 2, 3 evaluates to False and according the Table 4.5 the result of logical and, or operation with both the operands as False conditions will also be False, as it is verified from the output as well. Let's see the second case in the next example:

```
1 a, b = 10, 20
2 operand1 = a != b
3 operand2 = a > b
4 print(f'{operand1} and {operand2} => {operand1 and operand2}')
5 print(f'{operand1} or {operand2} => {operand1 or operand2}')
```

True and False => False
True or False => True

For the logical and operator if any of the operand represent a **False** value, the outcome of the logical and operator will be **False**. But for the logical or operator if any of the operand evaluates to **True**, then the outcome of the logical or operator will be **True**.

In the above code, the logical operators 'and' and 'or' are used with two operands 'operand1' and 'operand2'. The value of 'operand1' is assigned as the result of the expression 'a != b', which is True since the value of 'a' is not equal to the value of 'b'. The value of 'operand2' is assigned as the result of the expression 'a > b', which is False since the value of 'a' is not greater than the value of 'b'.

The first print statement (at line 4) uses the logical 'and' operator to evaluate the expression 'operand1 and operand2', which is **False** since 'operand2' is **False**. The second print statement (at line 5) uses the logical 'or' operator to evaluate the expression 'operand1 or operand2', which is **True** since 'operand1' is **True**. Let's test the final condition as both the operand as **True**.

```

1 a, b = 10, 20
2 operand1 = a != b
3 operand2 = a < b
4 print(f'{operand1} and {operand2} => {operand1 and operand2}')
5 print(f'{operand1} or {operand2} => {operand1 or operand2}')
```

```

True and True => True
True or True => True
```

If both the operands of logical and, or operators are True then, final output of operands are also True, as it can be verified from the output of above code. For a programmer, it is important to understand the evaluation order of the operands used with logical and, or operators. In Python, logical and, or operators are evaluated from left to right, as follows:

- When using the logical and operator, if the first operand evaluates to False, the second operand is not evaluated because the expression is already known to be False.
- When using the logical or operator, if the first operand evaluates to True, the second operand is not evaluated because the expression is already known to be True.

The logical not operator in Python is represented by the "not" keyword. It takes a single operand and returns the opposite Boolean value. It returns the inverted truth value for the operand and its general syntax is as follows:

not operand

Here's an example that uses the logical not operator:

```

1 x, y = 10, 5
2
3 # Evaluates to False, because x is greater than y
4 result1 = x < y
5 # Evaluates to True, because result1 is False
6 result2 = not result1
7 print(result1) # False
8 print(result2) # True
```

The values of **x** and **y** is compared using **<** operator, which returns a Boolean value of False because **x** is greater than **y**. Then apply the logical **not** operator to this result using the "**not**" keyword, which returns the opposite Boolean value of **True**. So, the result of **result2** is **True**, because it is the opposite of the original comparison result.

4.3.1 How does Python interpreter consider any value as truthy or falsy?

In Python, any value can be evaluated as either **True** or **False**, depending on whether it represents a "truthy" or "falsy" value. The truth value of an object in Python is determined by its Boolean context, which means the context in which the object is used in a Boolean expression. In general, the following values are considered "falsy" in Python: All the logical operators work with the truth value of their operands – but what exactly is a truth value?

We know that the bool type True represents True and False represents False. Python considers zero to be False and all other numbers, irrespective if they are positive or negative, are considered to be True.

```
>>> print(bool(8))
True
>>> print(bool(0))
False
>>> print(bool(-8))
True
```

In the given example we have checked what type of values are considered as True and False. We can see the output that 8 and -8 considered as True and zero (0) considered as False. In python most of the in-built types considered as False:

- constants defined to be False: None and False.
- zero of any numeric type: 0, 0.0, 0j, Decimal(0), Fraction(0, 1)
- empty sequences and collections: "", (), [], {}, set(), range(0)

Operations and built-in functions that have a Boolean result always return 0 or False for false and 1 or True for true, unless otherwise stated. (Important exception: the Boolean operations or, and always return one of their operands.)

4.3.2 Short-circuiting of logical operators

Logical **and**, **or** operators in Python are short-circuited which means they evaluate only the bare minimum required to get the correct result. For example:

```
>>> 3 < 2 and 4 != 1 and 3 > 1
False
```

Expression `3 < 2` evaluates to False, we know that the output of **and** operator is also False. Then does it make sense to evaluate the remaining expression `4 != 1 and 3 > 1`? Nope it does not make sense, and Python doesn't do it either. It starts evaluating from left to right, as soon as an expression is False, the "and" evaluates to False, skipping the execution of the remaining operands. The same thing happens for the "or" operator:

```
>>> 3 > 2 or 4 != 1 or 3 > 1
True
```

Expression `3 > 2` evaluates to True, then immediately the output of the "or" expression becomes True, ignoring the remaining part of the expressions. This saves spending unnecessary time on evaluating expressions whose output won't affect the final output of the expression anyway.

4.4 Bitwise Operator

A bitwise operator operates on individual bits of a number, rather than whole number. In computers information is stored in binary as 0s and 1s. These operators perform logical operations on these bits directly. The detailed list of bitwise operates are listed in Table 4.6, out of these bitwise NOT (~) requires one operand.

Table 4.6 Bitwise Operators

Operator	Example	Meaning
----------	---------	---------

&	a & b	bitwise and
 	a b	bitwise or
^	a ^ b	bitwise xor (exclusive or)
~	~a	bitwise not
<<	a << b	bitwise left shift
>>	a >> b	bitwise right shift

If you're coming from another programming language such as Java, then you'll immediately notice that Python is missing the unsigned right shift operator denoted by three greater-than signs (`>>>`). This has to do with how Python represents integers internally. Since integers in Python can have an infinite number of bits, the sign bit doesn't have a fixed position. In fact, there's no sign bit at all in Python.

All binary bitwise operators have a corresponding compound operator that performs an augmented assignment, as listed in Table 4.7:

Table 4.7 Compound Operations with Bitwise Operators

Operator	Example	Equivalent to
&=	A &= b	A = a & b
 =	A = b	A = a b
^=	A ^= b	A = a ^ b
<<=	A <<= b	A = a << b
>>=	A >>= b	A = a >> b

4.4.1 Why Binary Systems are used?

There are an infinite number of ways to represent numbers. Since ancient times, people have developed different notations, such as Roman numerals and Egyptian hieroglyphs. Most modern civilizations use positional notation, which is efficient, flexible, and well suited for doing arithmetic.

A notable feature of any positional system is its base, which represents the number of digits available. People naturally favor the base-ten numeral system, also known as the decimal system, because it plays nicely with counting on fingers.

Computers, on the other hand, treat data as a bunch of numbers expressed in the base-two numeral system, more commonly known as the binary system. Such numbers are composed of only two digits, zero and one.

For example, the binary number 10011100_2 is equivalent to 156_{10} in the base-ten system. Because there are ten numerals in the decimal system—zero through nine—it usually takes fewer digits to write the same number in base ten than in base two.

The binary system requires more storage space than the decimal system but is much less complicated to implement in hardware. While you need more building blocks, they're easier to make, and there are fewer types of them. That's like breaking down your code into more modular and reusable pieces.

More importantly, however, the binary system is perfect for electronic devices, which translate digits into different voltage levels. Because voltage likes to drift up and down due to various kinds of noise, you want to keep sufficient distance between consecutive voltages. Otherwise, the signal might end up distorted.

By employing only two states, you make the system more reliable and resistant to noise. Alternatively, you could jack up the voltage, but that would also increase the power consumption, which you definitely want to avoid.

4.4.2 How do Computers uses Binary?

Before any piece of information can be reproduced in digital form, you have to break it down into numbers and then convert them to the binary system. For example, plain text can be thought of as a string of characters. You could assign an arbitrary number to each character or pick an existing character encoding such as ASCII, ISO-8859-1, or UTF-8.

In Python, strings are represented as arrays of Unicode code points. To reveal their ordinal values, call **ord()** on each of the characters:

```
>>> msg = 'Hello'
>>> for character in msg:
...     print(f'{character} = {ord(character)}')
...
H = 72
e = 101
l = 108
l = 108
o = 111
```

The resulting numbers uniquely identify the text characters within the Unicode space, but they're shown in decimal form. To get binary version of any number we can use the **bin()**.

```
>>> msg = 'Hello$'
>>> for character in msg:
...     print(f'{character} = {bin(ord(character))}')
...
H = 0b1001000
e = 0b1100101
l = 0b1101100
l = 0b1101100
o = 0b1101111
$ = 0b100100
```

Notice that bit-length, which is the number of binary digits, varies greatly across the characters. The dollar sign (\$) requires six bits, while the rest of the characters can comfortably fit on seven bits. We can also check the number of bits of an integer using **bit_length()**.

```
>>> msg = 'Hello$'
>>> for character in msg:
...     print(f'{character} = {bin(ord(character))}, \
...     bit_length = {ord(character).bit_length()}')
...
H = 0b1001000, bit_length = 7
e = 0b1100101, bit_length = 7
l = 0b1101100, bit_length = 7
l = 0b1101100, bit_length = 7
o = 0b1101111, bit_length = 7
$ = 0b100100, bit_length = 6
```

Variable bit-lengths are problematic. If you were to put those binary numbers next to one another on an optical disc, for example, then you'd end up with a long stream of bits without clear boundaries

between the characters.

One way of knowing how to interpret this information is to designate fixed-length bit patterns for all characters. In modern computing, the smallest unit of information, called an octet or a byte, comprises eight bits that can store 256 distinct values.

4.4.3 Bitwise AND

The bitwise AND operator (&) performs logical conjunction on the corresponding bits of its operands. For each pair of bits occupying the same position of the two numbers, it returns 1 only when both bits are switched on. Check the Table 4.8, for bitwise operations:

Table 4.8 Working process of Bitwise AND Operator

Operand1 (bit)	Operand2 (bit)	Operand1 & Operand2
0	0	0
1	0	0
0	1	0
1	1	1

The resulting bit pattern is an intersection of the operator's arguments. It has two bits turned on in the positions where both operands are ones. In all other places, at least one of the inputs has a zero bit. Arithmetically, this is equivalent to a product of two bit values. You can calculate the bitwise AND of numbers **a**, **b** by multiplying their bits at every index **i**:

$$(a \& b)_i = a_i \times b_i$$

A bit having 1 multiplied by 1 gives 1, but anything multiplied by zero will always result in zero. Alternatively, you can take the minimum of the two bits in each pair. Notice that when operands have unequal bit-lengths, the shorter one is automatically padded with zeros to the left. See the coding example, given next.

```
>>> a, b = 16, 8
>>> c = a & b
>>> print(f'{bin(a)} & {bin(b)} = {bin(c)}')
0b10000 & 0b1000 = 0b0
>>> print(f'{a} & {b} = {c}')
16 & 8 = 0
```

In the above example, we have first printed the output in binary and then in decimal form. At first both numbers will be represented to same bit length as 8 bits. Because decimal 16 and 8 requires 5 and 4 bits respectively. During processing CPU will read byte by bytes, both the numbers need to be represented in 8 bits form and then bitwise AND will be applied over each pair of bits, as shown below:

```

  0  0  0  1  0  0  0  0
&  0  0  0  0  1  0  0  0
  -----
  0  0  0  0  0  0  0  0
```

4.4.4 Bitwise OR

The bitwise OR operator (`|`) performs logical disjunction. For each corresponding pair of bits, it returns 1, if at least one of the bit is switched on. The resulting bit pattern is a union of the operator's arguments. It turned on a bit where either of the operands has a bit as 1. Only a combination of two zeros gives a zero in the final output. See the process listed in Table 4.9.

Table 4.9 Working Process of Bitwise OR Operator

Operand1 (bit)	Operand2 (bit)	Operand1 Operand2
0	0	0
1	0	1
0	1	1
1	1	1

The arithmetic behind it is a combination of a sum and a product of the bit values. To calculate the bitwise OR of numbers **a**, **b**, you need to apply the following formula to their bits at every index **i**, and see the working of bitwise OR, in the given coding example.

$$(b)_i = a_i + b_i - (a_i \times b_i)$$

```
>>> a, b = 16, 8
>>> c = a | b
>>> print(f'{bin(a)} | {bin(b)} = {bin(c)}')
0b10000 | 0b1000 = 0b11000
>>> print(f'{a} | {b} = {c}')
16 | 8 = 24
```

At first both numbers will be represented to same bit length as 8 bits. Then bitwise OR will be applied over each pair of bits, as shown below:

```

  0  0  0  1  0  0  0  0
|  0  0  0  0  1  0  0  0
-----
  0  0  0  1  1  0  0  0
```

4.4.5 Bitwise XOR

The name XOR stands for “exclusive or” since it performs exclusive disjunction on the bit pairs. In other words, every bit pair must contain opposing bit values to produce a one:

Table 4.10 Working of Bitwise XOR Operator

Operand1 (bit)	Operand2 (bit)	Operand1 ^ Operand2
0	0	0
1	0	1
0	1	1
1	1	0

Visually, it's a symmetric difference of the operator's arguments. In Table 4.10, three bits switched on in the result where both numbers have different bit values. Bits in the remaining positions cancel out because they're the same.

Similarly to the bitwise OR operator, the arithmetic of XOR involves a sum. However, while the bitwise OR clamps values at one, the XOR operator wraps them around with a sum modulo two:

$$(a \wedge b)_i = (a_i + b_i) \bmod 2$$

Modulo is a function of two numbers—the dividend and the divisor—that performs a division and returns its remainder. In Python, there's a built-in modulo operator denoted with the percent sign (%).

```
>>> a, b = 34, 12
>>> c = a ^ b
>>> print(f'{bin(a)} ^ {bin(b)} = {bin(c)}')
0b100010 ^ 0b1100 = 0b101110
>>> print(f'{a} ^ {b} = {c}')
34 ^ 12 = 46
```

At first both numbers will be represented to same bit length as 8 bits. Then bitwise XOR applied over each pair of bits, as shown below:

	0	0	1	0	0	0	1	0
^	0	0	0	0	1	1	0	0
	0	0	1	0	1	1	1	0

4.4.6. Bitwise NOT

The last bitwise logical operator is the bitwise NOT operator (~), which takes only one operand, making it the only unary bitwise operator. It performs logical negation on a given number by flipping all of its bits:

~	0	0	1	0	0	0	1	0
	1	1	0	1	1	1	0	1

The inverted bits are a complement to one, which turns zeros into ones and ones into zeros. It can be expressed arithmetically as the subtraction of individual bit values from one:

$$\sim a_i = 1 - a_i$$

While the bitwise NOT operator seems to be the most straightforward of them all, you need to exercise extreme caution when using it in Python. Everything you've read so far assumes that numbers are represented with unsigned integers.

Note: Unsigned data types don't let you store negative numbers such as -273 because there's no space for a sign in a regular bit pattern. Trying to do so would result in a compilation error, a runtime exception, or an integer overflow depending on the language used.

Although there are ways to simulate unsigned integers, Python doesn't support them natively. That

means all numbers have an implicit sign attached to them whether you specify one or not. This shows when you do a bitwise NOT of any number:

```
>>>a = 156
>>>bin(a)
'0b10011100'
```

Now if we flip all the bits of **a** using bitwise NOT, the bits of a will be something like '0b01100011', which is 9910 in decimal, but Python tells different story:

```
>>>~a
-157
```

Remember, one's complement of any positive number will give a negative number, because of change in sign bit. In a computer system, the left most bit is reserved for preserving the sign of any number. If the sign is '+' then left most bit will be 0, else 1 for negative numbers. To store any negative value computer systems uses two's complement of the number and follows the steps given below:

- Extracts all the bits except the sign bit from the resultant binary string, that we received after applying one's complement. So, we have '0b01100011', we didn't consider the sign bit, which is already 1.
- In the next step, we need to find the one's complement of '0b01100011', which will be '0b10011100'.
- In the last step, we will perform binary 1 addition with the binary number received in previous step, which will be '0b10011101' means 15710.
- In the complete process the sign bit remains unchanged and the output is produced after converting the binary string to decimal with a prefixed negative sign.

```
>>>print(0b10011101)
157
```

4.4.7 Bitwise Shift Operators

Bitwise shift operators are another kind of tool for bit manipulation. They let you move the bits around, which will be handy for creating bitmasks later. In the past, they were often used to improve the speed of certain mathematical operations.

Left Shift

The bitwise left shift operator (<<) moves the bits of its first operand to the left by the number of places specified in its second operand. It also takes care of inserting enough zero bits to fill the gap that arises on the right edge of the new bit pattern. Let's see an example,

```
>>> a = 12
>>> a << 1
24
```

In the above code we are shifting the bits of **a** by 1 position towards left and the value 12 will be multiplied by 2 and 24 will be saved into b. So, the print statement will print 24. Let's see a pictorial view of the above example:

Bit position	7	6	5	4	3	2	1	0
Operand 1	0	0	0	0	1	1	0	0
After left shift	0	0	0	1	1	0	0	0

Now the second operand holds the value 1. So, every bits will change their position by 1 towards left. So the new number after shifting of 1 bit is 11000 and the decimal of this number is 24. Shifting a single bit to the left by one place doubles its value. Moving bits in two places to the left will quadruple the resulting value. In general, shifting bits to the left corresponds to multiplying the number by a power of two, with an exponent equal to the number of places shifted:

$$a \ll n = a \times 2^n$$

The left shift used to be a popular optimization technique because bit shifting is a single instruction and is cheaper to calculate than exponent or product. Today, however, compilers and interpreters, including Python's, are quite capable of optimizing your code behind the scenes.

Right Shift

The bitwise right shift operator ($\gg$) is analogous to the left one, but instead of moving bits to the left, it pushes them to the right by the specified number of places. The rightmost bits always get dropped. Let's take an example:

```
>>> a = 39
>>> bin(a)
'0b100111'
>>> b = a >> 1
>>> bin(b)
'0b10011'
```

The above code demonstrates the bitwise right shift ($\gg$) operation. Unlike arithmetic division, a right shift by 1 position on a binary number is equivalent to dividing by 2, its decimal value. The code assigns the shifted value to b. However, it's important to note that the right shift operation discards the least significant bits (those on the right). This effectively truncates the division result, discarding any fractional component.

Bit position	7	6	5	4	3	2	1	0
Operand1	0	0	1	0	0	1	1	1
after right shift	0	0	0	1	0	0	1	1

Every time you shift a bit to the right by one position, you halve its underlying value. Moving the same bit by two places to the right produces a quarter of the original value, and so on. When you add up all the individual bits, you'll see that the same rule applies to the number they represent:

Expression	Binary Value	Decimal Value
a	10011101	157
a >> 1	1001110	78
a >> 2	100111	39
a >> 3	10011	19

Halving an odd number such as 157 would produce a fraction. To get rid of it, the right shift operator automatically floors the result. It's virtually the same as a floor division by a power of two:

$$a \gg n = \lfloor \frac{a}{2^n} \rfloor$$

The bitwise right shift operator and the floor division operator both work the same way, even for negative numbers. However, the floor division lets you choose any divisor and not just a power of two. Using the bitwise right shift was a common way of improving the performance of some arithmetic divisions. You might be wondering what happens when you run out of bits to shift. For example, when you try pushing by more places than there are bits in a number:

```
>>> a = 3
>>> a >> 5
0
```

In this example **a**, is right shifted by 5 bits. But the binary of 3 is 11_2 , which has two bits only. So in this case result will be always zero. Once there are no more bits switched on, you're stuck with a value of zero. Zero divided by anything will always return zero. However, things get trickier when you right shift a negative number because the implicit sign bit gets in the way:

```
>>> a = -2
>>> a >> 5
-1
```

The rule of thumb is that, regardless of the sign, the result will be the same as a floor division by some power of two. The floor of a small negative fraction is always minus one, and that's what you'll get.

4.5 Assignment Operator

You have seen that a single equal sign (**=**) is used to assign a value to a variable. It is, of course, perfectly viable for the value to the right of the assignment to be an expression containing other variables:

```
>>> a = 10
>>> b = 20
>>> c = a * 5 + b
>>> c
70
```

In fact, the expression to the right of the assignment can include references to the variable that is being assigned to:

```
>>> a = 10
>>> a = a + 5
>>> a
15
>>> b = 20
>>> b = b * 3
>>> b
60
```

The first example is interpreted as “**a** is assigned the current value of **a** plus 5,” effectively increasing the value of **a** by 5. The second reads “**b** is assigned the current value of **b** times 3,” effectively increasing the value of **b** threefold. Of course, this sort of assignment only makes sense if the variable in expression **a + 5** or **b * 3** has already previously been assigned a value:

```
>>> var = var / 3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'var' is not defined.
```

At first line, **var** has been referred to in the expression **var / 3**, which has not defined yet. So doing this is illegal in Python, Else raised a **NameError**.

4.5.1 Shorthand Assignment

A shorthand assignment operator, also called a compound assignment operator, is a way to combine an assignment operation (using the equals sign '=') with another arithmetic or bitwise operator. Python supports a shorthand augmented assignment notation for these arithmetic and bitwise operators as listed below:

Table 4.11 List of Arithmetic and Bitwise Operators Supports shorthand assignment

Arithmetic	Bitwise
+	&
-	
*	^
/	>>
%	<<
//	
**	

For the above listed operators following two expressions are equivalent:

variable operator= expression

variable = variable operator expression

In the above syntax “operator” represent any of the operators from arithmetic and bitwise listed in Table 4.11. Look at the next example, where we have highlighted three types of assignment statement and produce same result for **x** and **y**.

```
>>> x = y = 12 #augmented assignment
>>> x += 2 #compound assignment
>>> y = y + 2 #standard assignment
>>> x, y
(14, 14)
```

4.6 Membership Operator

Python offers two membership operators to check or validate the membership of a value. It tests for membership in a sequence, like a list, tuple, set or string.

Table 4.12 Details of Membership Operators

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	X in Y
not in	Returns True if a sequence with the specified value is not present in the object	X not in Y

The ‘in’ operator is used to check if a character/ substring/ element exists in a sequence or not. Evaluate to True if it finds the specified element in a sequence otherwise, False. For example,

```
>>> l = [2, 3, 4]
>>> 2 in l
True
>>> 10 in l
False
>>> s = (2, 3)
>>> 2 in s
True
>>> 4 in s
False
```

The ‘not in’ operator is the opposite of ‘in’. It returns True, if the specified value is not found in the sequence. Otherwise, it returns False.

```
>>> colors = ["red", "green", "blue"]
>>> "pink" not in colors
True
>>> letters = "Python"
>>> "p" not in letters
True
>>> "P" not in letters
False
```

4.7 Identity Operator

Identity operators compare the memory locations of two objects. There are two Identity operators explained below:

Operator	Description	Example
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.	x is y, here is results in 1 if id(x) equals id(y)
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.	x is not y, here is not results in 1 if id(x) is not equal to id(y).

In python every object has an identity, a type and a value. An object's identity never changes once it has been created; you may think of it as the object's address in memory. The 'is' operator compares the identity of two objects; the id() function returns an integer representing its identity.

```
>>> a = 20
>>> b = 20
>>> a is b
True
>>> id(a), id(b)
(140715626501512, 140715626501512)
```

This code snippet checks if two variables, a and b, refer to the same object in memory and not just having the same value. The line **a is b**, uses the is operator, checks if a and b refer to the exact same object in memory. In this case, since both variables are assigned the same small integer value (20), Python might optimize by storing only one copy of the value and making both a and b point to that single copy. Both a and b refer to the same memory block as confirmed using **id()**.

For small integers like 20, Python might have an internal pool of pre-allocated memory for these values to improve performance. Assigning the same value to multiple variables allows Python to simply reference the existing object instead of creating a new one in memory for each variable. This behavior depends on the data type, value, and potentially even Python implementation details. Here are some scenarios where **a is b**, might return False:

- **Larger integers or other data types:** For larger integers or other data types like strings or lists, Python might not pre-allocate memory and would likely create separate objects for each variable assignment.
- **Explicit object creation:** If you explicitly create objects using functions or constructors (`a = list(range(5))`, `b = list(range(5))`), even if they have the same content, they will be separate objects.

In the next example we are demonstrating that two objects will refer different memory address.

```
>>> a = 20.5
>>> b = 20
>>> a is b
False
>>> id(a), id(b)
(2424282604688, 140716333962632)
```

In the above example, `a` and `b` are two different type objects, `a` as float and `b` as int. The identity operator return False, as `a` and `b`, referring to different memory blocks. It is also true for large integers. Let us try `is` operator with mutable data types such as list.

```
>>> list1, list2 = [], []
>>> list3 = list1
>>> list1 is list2
False
>>> list1 is list3
True
```

In this code, we have created two empty list `list1` and `list2` and then assigned `list1` to `list3`. The identity of `list1` and `list2` is different even though they are referring empty list. But the identity of `list3` and `list1` are same, because we have assigned the value of `list1` to `list3`. If we used chained assignment instead of multiple assignment then the result will change, and all the list objects will refer to the same memory block.

```
>>> list1 is list2
True
>>> list1 is list3
True
>>> list3 is list2
True
```

`is not` operator evaluates True, if both variables are not the same object. See the example given below:

```
>>> l1 = [1,2,3]
>>> l2 = [1,2,3]
>>> l3 = l1
>>> l1 is not l2
True
>>> l1 is not l3
False
```

The behavior of **is** and **is not** operator also depends on the type of object. Python has two types of objects (i) mutable and (ii) immutable. Details of the mutable and immutable types are discussed in Chapter 3. Types affect almost all aspects of object behavior. Even the importance of object identity is affected in some sense: for immutable types, operations that compute new values may return a reference to any existing object with the same type and value, while for mutable objects this is not allowed.

For example, **a = 1** and **b = 1**, **a** and **b** may or may not refer to the same object with the value one, depending on the implementation of python, but after **c = []**; **d = []**, **c** and **d** are guaranteed to refer to two different, unique, newly created empty lists.

4.8 Operator Precedence and Associativity

While dealing with operators in Python, we must know about the concept of Python operator precedence and associativity as these determine the priorities of the operator otherwise, we'll see unexpected outputs.

4.8.1 Operator Precedence

Imagine an expression with multiple operators like addition, multiplication, and division. Precedence determines which operations are evaluated first. Operators with higher precedence are evaluated before those with lower precedence. This ensures the expression is evaluated in the intended order. Check the example given below:

```
>>> a = 4 + 5 * 3
>>> a
19
```

Here multiplication operator is higher in precedence. So, $5 * 3$ will be evaluated first and 15 will be added with 4. We can also change the order of evaluation using a pair of brackets, as shown below:

```
>>> a = 4 / (5 + 3)
>>> a
0.5
```

In the above code addition is performed over division, because the expression **5+3** surrounded by a pair of parentheses. Here is the order of precedence of the Python operators you have seen so far, from lowest to highest:

Table 4.13 Order of Operator Precedence and Associativity

Precedence	Operator	Description	Associativity
Highest	()	Parenthesis	Left to Right
	x[index], x[index : index]	Subscripting, slicing	Left to Right
	**	Exponentiation	Right to Left
	+x, -x, ~x	unary positive, unary negation, bitwise negation	Right to Left
	*, /, //, %	multiplication, division, floor division, modulo	Left to Right
	+, -	addition, subtraction	Left to Right
	<<, >>	bitwise shift operators	Left to Right
	&	Bitwise AND	Left to Right
	^	Bitwise XOR	Left to Right
		Bitwise OR	Left to Right
	In, not in, ==, !=, <, <=, >, >=, is, is not	Comparisons and identity operators	Left to Right
	not x	Logical not	Right to Left
	And	Logical and	Left to Right
	Or	Logical or	Left to Right
Lowest			

Operators at the top of the Table 4.13 have the highest precedence, and those at the bottom of the table have the lowest. Any operators in the same row of the table have equal precedence. That's why multiplication is performed first, as it is higher precedence than addition.

Similarly, in the next example, 3 raised to the power of 4, will be evaluated first, and then multiplications are carried out in order from left to right ($2 * 81 * 5 = 810$):

```
>>> 2 * 3 ** 4 * 5
810
```

Operator precedence can be overridden using parentheses. Expressions in parentheses are always performed first, before expressions that are not parenthesized. Thus, the following happens:

```
>>> 20 + 4 * 10
60
>>> (20 + 4) * 10
240
```

The two expressions generate different results because of operator precedence in Python. Operator precedence defines the order in which operations are evaluated within an expression. In the first expression, multiplication (*) has higher precedence than addition (+). Therefore, $4 * 10$ is evaluated first (40). Then, 40 is added to 20, resulting in 60. In the second expression parentheses () force the evaluation of $20 + 4$ first (24), then 24 is multiplied by 10, resulting in 240.

There is nothing wrong with making liberal use of parentheses, even when they aren't necessary to change the order of evaluation. In fact, it is considered good practice, because it can make the code more readable, and it relieves the reader of having to recall operator precedence from memory. Consider the following:

```
>>> a, b = 30, 10
>>> (a < 40) and (b > 5)
True
```

Use of parentheses is fully unnecessary, as the comparison operators have higher precedence than **logical and** does and would have been performed first anyhow. But some might consider the intent of the parenthesized version more immediately obvious than this version without parentheses:

```
>>> a < 40 and b > 5
True
```

On the other hand, there are probably those who would prefer the latter; it's a matter of personal preference. The point is, you can always use parentheses if you feel it makes the code more readable, even if they aren't necessary to change the order of evaluation.

4.8.2 Operator Associativity

If an expression contains two or more operators with the same precedence, then operator associativity is used to evaluate the expression. It can either be Left to Right or from Right to Left. Next example shows multiple operators with same precedence. To solve this expression, we have to understand how operator associativity works. Details order of operator associativity is listed in Table 4.13. Some of the coding examples are listed next to understand the working of operator associativity.

```
>>> 4 / 5 * 3
2.4000000000000004
```

In the above expression division operation is evaluated first, which results 0.8 and multiplied with 3. Due to the rule of associativity this expression is evaluated from left to right. Associativity of operators are considered when in a given expression more than one operator are used with same precedence. The right to left associativity can be seen in the next example.

```
>>> 2 ** 3 ** 2
512
```

In the above expression, $3 ** 2$ will be evaluated first, which results in 9 followed by $2 ** 9$ is calculated, using right to left associativity. In python, all the unary (operates over single operand) operators followed right to left associativity, as listed in Table 4.13.

4.8.3 Non-Associative Operators

In Python, non-associative operators are operators where the order of evaluation within an expression doesn't follow the left-to-right or right-to-left rule that applies to most operators. This means you cannot chain them together and expect a well-defined result. Here are the key points about non-associative operators in Python:

- **No defined order of evaluation:** Unlike operators with precedence (like addition and multiplication), non-associative operators have no set order in which they are evaluated. This can lead to unexpected results if you try to chain them together.
- **Limited examples:** There are only a few non-associative operators in Python, primarily assignment operators and some comparison operators used in a specific way.

Operators like `=`, `+=`, `-=`, etc., are non-associative. You cannot combine them like `x = y = z += 1`. This would result in a syntax error.

```
a = b = c += 1
File "<stdin>", line 1
    a = b = c += 1
                ^^
```

SyntaxError: invalid syntax

While comparison operators (`<`, `>`, `<=`, `>=`, `==`, `!=`) themselves are associative (evaluated left-to-right), using them in a chain without parentheses is non-associative. For example, `a < b < c` is not equivalent to `(a < b)` and `(b < c)`. It's generally discouraged to chain.

Key Takeaway's

- Logical operators used to combine conditional statements: `and`, `or`, and `not`.
- Membership Operators are used to test if a sequence is presented in an object: `in`, `not in`.
- Identity Operators are used to compare the memory locations of two objects: `is`, `is not`.
- Operators with higher precedence are evaluated before operators with lower precedence.
- When operators have the same precedence, associativity determines the order of evaluation. Most operators in Python have left-to-right associativity, meaning they are evaluated from left to right.
- Use parentheses `()` to override the default precedence and associativity.
- The exponentiation operator `**` has right-to-left associativity, which is an exception to the common left-to-right rule.
- Unary plus `+` and minus `-` (as in `+x`, `-x`) have higher precedence than the standard arithmetic operators.
- Multiplicative operators (`*`, `/`, `//`, `%`) have higher precedence than additive operators (`+`, `-`).
- All comparison operators have the same precedence, which is lower than that of arithmetic and bitwise operators.
- The logical `not` has higher precedence than `and`, which has higher precedence than `or`

*****END OF CHAPTER*****